Semester 01 2025/2026

Subject : Database Programming (SECP3623)
Section : Section 1
Task : Project II
Due : 19th January 2026 (10%)

Name	**Matric No.**
BRENDAN CHIA YAN FEI	A23CS0211
GUI KAH SIN	A23CS0080
SABRINA HENG WEI QI	A23CS0265
LAU YAN KAI	A23CS0098

Presentation URL: https://youtu.be/g80_VspGgn8

Introduction

1.1 Project Background

As professional networking and technical workshops grow, we need better ways to manage event data. Traditional databases often struggle with event schedules and the different types of information found in technology workshops. This project uses MongoDB Atlas to create a flexible NoSQL database called EventManagementDB. By using a document-oriented model, the system is designed to handle complex relationships between events, venues, organisers, and attendees with high scalability and flexibility.

1.2 Project Objectives

The primary goal of this project is to develop a robust and flexible database for event management using MongoDB Atlas. First, the project focuses on Schema Design, where we implement a structure that combines embedding and referencing to optimize how data is retrieved and stored. Second, through CRUD Implementation, the team demonstrates proficiency in core operations, specifically the insertion, reading, updating, and deletion of complex documents. Third, we address Performance Optimization by applying indexing strategies to reduce query latency and ensure fast event searches. Finally, the project utilizes Advanced Analytics via the MongoDB Aggregation Framework to generate deep insights, such as tracking attendee counts and analyzing revenue for each event.

1.3 Team Members and Roles

Team Member	Roles
Brendan Chia Yan Fei A23CS0211	- Design MongoDB collections - Schema Design - Introduction - Explanation of Collections
Gui Kah Sin A23CS0080	- Analyse NoSQL security, injection risk and access control - Identify database limit - Conclusion
Sabrina Heng Wei Qi A23CS0265	- Basic Operation - Advance Filtering, Updates, Deletes - CRUD Implementation

Lau Yan Kai A23CS0098	- Indexing, Aggregation, Sorting - Advanced operations - Explain performance improvement from indexing

NoSQL Data Model

2.1 Introduction to the Event Management Schema

For this project, the chosen domain is Event Management. This system manages the relationship between organizers who create events, the venues where they take place, and the attendees who register for them. Specifically, our database handles the backend requirements for data storage, registration tracking, and ticket inventory.

MongoDB, a NoSQL document model is ideal for this domain because it prioritizes flexibility and performance over rigid structure. By utilizing schema flexibility, the system can easily store varying data fields for different event types within the same collection without the constraints of a traditional table. Furthermore, its hierarchical data handling capabilities allow for embedded documents, enabling ticket tiers or schedules to be stored directly inside an event document to eliminate the need for complex joins. This model also provides scalability for high traffic, leveraging MongoDB's distributed nature to handle the sudden surges in database requests typical during major ticket launches. Finally, the natural representation of JSON-like BSON documents maps directly to modern application code, which simplifies the development and execution of the required CRUD operations.

2.2 List and Explanation of Collections

Our event management system is built using four core collections to ensure a structured yet flexible data organisation.

Collection	Explanation
Events	Stores the primary details of each event, including titles, descriptions, and scheduling information.
Attendees	Manages user profiles and registration data for individuals participating in events.
Organisers	Tracks the entities or individuals responsible for hosting and managing the events.
Venues	Stores location data, including capacity and facility details for the physical or virtual spaces where events occur.

2.3 Example JSON Documents

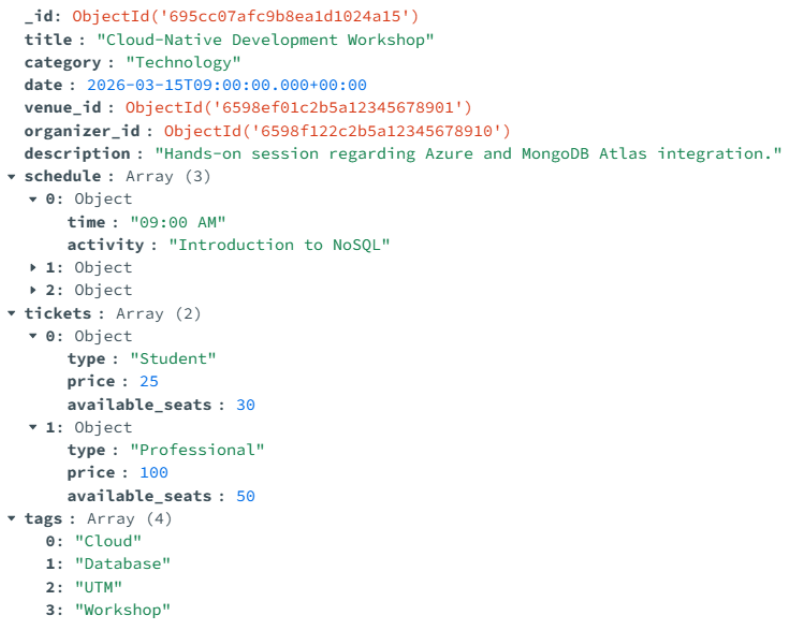


Figure 2.3.1 (Events collection)

1. **Referencing**: the venue_id and organizer_id fields store **ObjectId** references to documents in other collections . This allows the event to link to specific location and host data without duplicating that information within the event document itself.
2. **Embedded Arrays and Objects**: The schedule field is an array containing three embedded objects. The tickets field is an array of objects that embeds different ticket tiers. The tags field is a simple array of strings used for indexing and categorizing the event for easier searching

2.4 Explanation of Embedding vs Referencing

Embedded Array	Reference Field
venue_id : ObjectId('6598ef01c2b5a1 organizer_id : ObjectId('6598f122c1	▾ tickets : Array (2) ▾ 0: Object type : "Student" price : 25 available_seats : 30 ▾ 1: Object type : "Professional" price : 100 available_seats : 50

In our event management system , we applied both strategies to optimize performance and data integrity.

1. Embedding : We chose to embed ticket tiers and event schedules within the Events collection. This is highly effective because these details are almost always retrieved at the same time the event details are queried, reducing database read operations.
2. Referencing : We used manual references to link the events collection to the organisers and venues collections. because an organiser or a venue can be associated with hundreds of different events, referencing prevents data duplication and simplifies updates to the organiser's contact information or the venue's capacity.

2.5 Summary of Data Types and BSON Implementation

BSON Data Type	Application
objectid	used for primary keys (_id) and to create manual references between collections, such as linking an event to its organizer.
string	applied to all textual data fields, including event titles, category names, and descriptions.
date	used for temporal fields like the event start time to enable efficient time-based filtering and sorting.

number	utilized for numeric values such as ticket prices and the number of available seats.
array	used for multi-valued fields, such as the list of tags for searching or the collection of ticket objects within an event.
object	used to create embedded documents, allowing related data like specific ticket details to be stored directly inside the parent event document.

CRUD Implementation Summary

3.1 Explanation of insert, query, update, delete operations

This section will demonstrate the implementation of basic and advanced MongoDB operations across four collections, such as Attendees, Events, Organisers and Venues. The CRUD operations were designed to simulate realistic interactions within an event management system while demonstrating MongoDB's document-based data handling capabilities.

3.1.1 Insert Operation

CODE	OUTPUT
1. **insertOne()** - Use insertOne() operation to insert one new document into Attendees collection that has different types of data.	

```
db.Attendees.insertOne({
  name: "Poh Lok Yee",
  email: "lokyee@gmail.com",
  type: "Student",
  registered_events: [
    ObjectId("695cc07afc9b8ea1d1024a15")
  ],
  interests: ["Cloud","DevOps"]
})
```

```
{
  acknowledged: true,
  insertedId: ObjectId('6962178020e4e62c9ac28250')
}
```

```
_id: ObjectId('6962178020e4e62c9ac28250')
name : "Poh Lok Yee"
email : "lokyee@gmail.com"
type : "Student"
registered_events : Array (1)
  0: ObjectId('695cc07afc9b8ea1d1024a15')
interests : Array (2)
  0: "Cloud"
  1: "DevOps"
```

2. **insertMany() -** Use insertMany() operation to insert one than one new document into Events collection, by using this, it can increase efficiency as we don't need to insert same collections of data one by one.

```
db.Events.insertMany([
    {
      title: "AI Bootcamp 2026",
      category: "Techlonogy",
      date: ISODate("2026-06-01T09:00:00Z"),
      venue_id: ObjectId("6598ef01c2b5a12345678901"),
      organizer_id: ObjectId("6598f122c2b5a12345678913"),
      description: "Join us, to learn more on AI in future!",

      schedule: [
        {time: "08:30 AM", activity: "Regitration" },
        {time: "10.00 AM", activity: "Keynote: How AI changes future ?"},
        {time: "11.00 AM", activity: "Hands on session" },
        {time: "12.30 PM", activity: "Lunch Time" },
        {time: "2.00 PM", activity: "Photo session and Goodbye !"}
      ],

      tickets: [
        {type: "Student", price: 10, available_seats: 100},
        {type: "Non-Student", price: 25, available_seats: 100}
      ],
      tags: ["AI","ML","Workshop"]
    },
    {
```

```
      title: "Fintech Innovation Forum",
      category: "Finance",
      date: ISODate("2026-08-15T10:00:00Z"),
      venue_id: ObjectId("6598ef01c2b5a12345678902"),
      organizer_id: ObjectId("6598f122c2b5a12345678914"),
      description: "A high-level summit focusing on blockchain, digital banking, and the future of financial technology.",

    schedule: [
      { time: "08:30 AM", activity: "Registration & Coffee" },
      { time: "10:00 AM", activity: "Keynote: Central Bank Digital Currencies" },
      { time: "01:30 PM", activity: "Panel: Cybersecurity in Open Banking" },
      { time: "04:00 PM", activity: "Startup Pitch Competition" }
    ],

    tickets: [
      { type: "Early Access", price: 500, available_seats: 50 },
      { type: "Corporate Pass", price: 1200, available_seats: 200 },
      { type: "Exhibitor Pass", price: 3500, available_seats: 20 }
    ],
      tags: ["Blockchain","Finance"]
    }
  ]);
{
```

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6962251920e4e62c9ac28256'),
    '1': ObjectId('6962251920e4e62c9ac28257')
  }
}
```

```
_id: ObjectId('6962251920e4e62c9ac28256')
title : "AI Bootcamp 2026"
category : "Techlonogy"
date : 2026-06-01T09:00:00.000+00:00
venue_id : ObjectId('6598ef01c2b5a12345678901')
organizer_id : ObjectId('6598f122c2b5a12345678913')
description : "Join us, to learn more on AI in future!"
▸ schedule : Array (5)
▸ tickets : Array (2)
▸ tags : Array (3)
```

```
_id: ObjectId('6962251920e4e62c9ac28257')
title : "Fintech Innovation Forum"
category : "Finance"
date : 2026-08-15T10:00:00.000+00:00
venue_id : ObjectId('6598ef01c2b5a12345678902')
organizer_id : ObjectId('6598f122c2b5a12345678914')
description : "A high-level summit focusing on blockchain, digital banking, and the f…"
▸ schedule : Array (4)
▸ tickets : Array (3)
▸ tags : Array (2)
```

3.1.2 Query Operations

CODE	OUTPUT
1. find() - use find() to retrieve multiple documents from a collection based on specified conditions or to display all records.	

`> db.Attendees.find({type: "General"})`	`< {`
	`  _id: ObjectId('695cd174a392d4601b884da0'),`
	`  name: 'Eunice Lim',`
	`  email: 'eunice.lim@gmail.com',`
	`  type: 'General',`
	`  registered_events: [`
	`  ObjectId('695cc77ca392d4601b884d95')`
	`  ],`
	`  interests: [`
	`  'Robotics',`
	`  'Innovation'`
	`  ]`
	`}`
	`{`
	`  _id: ObjectId('695cd174a392d4601b884da3'),`
	`  name: 'Jason Lee',`
	`  email: 'jason.lee@gmail.com',`
	`  type: 'General',`
	`  registered_events: [`
	`  ObjectId('695cc07afc9b8ea1d1024a15'),`
	`  ObjectId('695cca0aa392d4601b884d96')`
	`  ],`
	`  interests: [`
	`  'Workshop',`
	`  'Security'`
	`  ]`
	`}`

2. findOne() - use findOne() operation to retrieve only one specific document based on specific conditions.	

`> db.Attendees.findOne({type: "General"})`	`< {`
	`  _id: ObjectId('695cd174a392d4601b884da0'),`
	`  name: 'Eunice Lim',`
	`  email: 'eunice.lim@gmail.com',`
	`  type: 'General',`
	`  registered_events: [`
	`  ObjectId('695cc77ca392d4601b884d95')`
	`  ],`
	`  interests: [`
	`  'Robotics',`
	`  'Innovation'`
	`  ]`
	`}`

3. $eq – use $eq to filter the Organizer that having the equal name that be specified

```
> db.Organisers.find({ org_name: { $eq: "UTM Faculty of Computing"}})
```

```
< {
    _id: ObjectId('6598f122c2b5a12345678910'),
    org_name: 'UTM Faculty of Computing',
    contact_email: 'info@computing.utm.my',
    website: 'https://computing.utm.my',
    rating: 4.9,
    specialization: [
      'Cloud Computing',
      'NoSQL Databases',
      'Software Engineering'
    ],
    established_year: 1981
  }
```

4. $gt – use $gt to find the Venue with capacity greater than 2000, $gt is a condition set for searching value greater

```
> db.Venues.find({ capacity: { $gt: 2000 } })
```

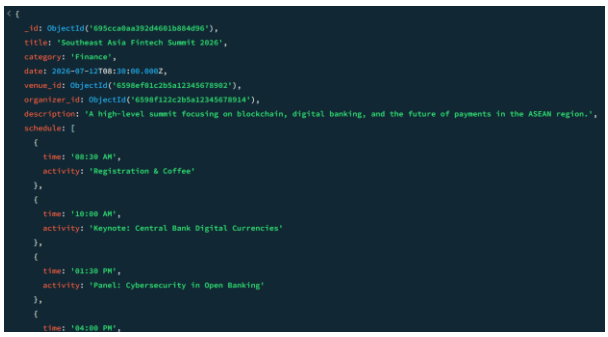
```
< {
    _id: ObjectId('6598ef01c2b5a12345678902'),
    name: 'Persada Johor International Convention Centre',
    address: 'Jalan Abdullah Ibrahim, 80000 Johor Bahru',
    capacity: 3000,
    facilities: [
      'Exhibition Hall',
      'Catering Service',
      'Parking',
      'Video Conferencing'
    ],
    contact_number: '+60 7-219 8888'
  }
  {
    _id: ObjectId('6598ef01c2b5a12345678903'),
    name: 'Kuala Lumpur Convention Centre (KLCC)',
    address: 'Kuala Lumpur City Centre, 50088 Kuala Lumpur',
    capacity: 5000,
    facilities: [
      'Auditorium',
      'Ballroom',
      'Digital Signage',
      '5G Ready'
    ],
    contact_number: '+60 3-2333 2888'
  }
```

5. **$lt -** use $gt to find the Venue with capacity less than 2000, $gt is a condition set for searching value lesser

```
> db.Events.find({ date: { $lt: ISODate("2026-07-01T00:00:00Z") } })
```

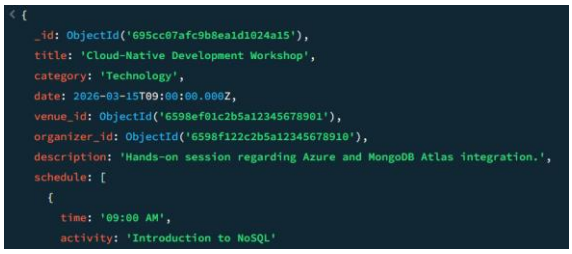
6. **$in –** use $in to match the events that having tags named “Blockchain” that store in Array

```
> db.Events.find({ tags: { $in: ["Blockchain"] } })
```



7. **$and –** use $and to apply multiple conditions together. To find events with both conditions, we use $and for both category and date set for searching.

```
> db.Events.find({
  $and: [
    { category: "Technology" },
    { date: { $gt: ISODate("2026-01-01T00:00:00Z") } }
  ]
})
```



8. **$or –** use $or to retrieve at least one condition matched. To find attendees that match one of the two conditions, type Student and Academic, we can use or. Then, the result will get at least one of the condition.

```
> db.Attendees.find({
    $or: [
      { type: "Student" },
      { type: "Academic" }
    ]
  })
< {
```

```
< {
    _id: ObjectId('695cd174a392d4601b884d9c'),
    name: 'Brendan Chia',
    email: 'yan04@graduate.utm.my',
    type: 'Student',
    registered_events: [
```

```
{
  _id: ObjectId('695cd174a392d4601b884d9e'),
  name: 'Siti Aminah',
  email: 'siti.a@utm.edu.my',
  type: 'Academic',
  registered_events: [
    ObjectId('695cc07afc9b8ea1d1024a15'),
```

9. **Array Queries –** This is use to query values stored inside array fields like tags, facilities

```
> db.Venues.find({ facilities: "High-speed WiFi" })
```

```
< {
    _id: ObjectId('6598ef01c2b5a12345678901'),
    name: 'UTM Convention Centre',
    address: 'Skudai, 81310 Johor Bahru, Johor',
    capacity: 1000,
    facilities: [
      'High-speed WiFi',
      'Projectors',
      'Sound System',
      'VIP Lounge'
    ],
    contact_number: '+60 7-553 3333'
  }
```

10. **Projection –** This is used to include or exclude specific fields from query results to return only necessary data.

```
> db.Attendees.find(
    {},
    { name: 1, email: 1, _id: 0 }
  )
< {
    name: 'Brendan Chia',
    email: 'yan04@graduate.utm.my'
  }
  {
    name: 'Sarah Tan',
    email: 'sarah.tan@techcorp.com'
  }
  {
    name: 'Siti Aminah',
    email: 'siti.a@utm.edu.my'
  }
  {
    name: 'Kevin Wong',
    email: 'kevin.wong@startup.io'
  }
```

3.1.3 Update Operations

CODE	OUTPUT

1. **updateOne() –** used to update a single matching document, such as modifying one attendee's profile

```
> db.Attendees.updateOne(
    { email: "siti.a@utm.edu.my" },
    { $set: { type: "Professional" } }
  )
```

```
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

```
_id: ObjectId('695cd174a392d4601b884d9e')
name : "Siti Aminah"
email : "siti.a@utm.edu.my"
type : "Professional"
▸ registered_events : Array (3)
▸ interests : Array (2)
```

2. **updateMany()** – used to update multiple data in one time that match a given condition across collections

```
> db.Organisers.updateMany(
    { verified_status: false },
    { $set: { verified_status: true } }
  );
```

```
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 0,
    modifiedCount: 0,
    upsertedCount: 0
  }
```

```
_id: ObjectId('6598f122c2b5a12345678914')
org_name : "SEA Finance Hub"
contact_email : "events@seafinance.com"
website : "https://seafinance.com"
rating : 4.5
▸ specialization : Array (3)
verified_status : true
```

3. **$set** – used to modify or add specific fields in database without affecting others

```
> db.Events.updateOne(
    { title: "AI Bootcamp 2026" },
    { $set: { category: "Education" } }
  )
```

```
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

```
_id: ObjectId('6962251920e4e62c9ac28256')
title : "AI Bootcamp 2026"
category : "Education"
date : 2026-06-01T09:00:00.000+00:00
venue_id : ObjectId('6598ef01c2b5a12345678901')
organizer_id : ObjectId('6598f122c2b5a12345678913')
description : "Join us, to learn more on AI in future!"
▸ schedule : Array (5)
▸ tickets : Array (2)
▸ tags : Array (3)
```

4. **$inc** – used to increase or decrease numeric values

```
> db.Organisers.updateOne(
    { org_name: "SEA Finance Hub" },
    { $inc: { rating: 0.2 } }
  )
```

```
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

```
_id: ObjectId('6598f122c2b5a12345678914')
org_name : "SEA Finance Hub"
contact_email : "events@seafinance.com"
website : "https://seafinance.com"
rating : 4.7
specialization : Array (3)
verified_status : true
```

5. **$push** – used to add new values to array fields

```
> db.Attendees.updateOne(
    { name: "Kevin Wong" },
    { $push: { interests: "Startups" } }
  )
```

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
_id: ObjectId('695cd174a392d4601b884d9f')
name : "Kevin Wong"
email : "kevin.wong@startup.io"
type : "Professional"
registered_events : Array (1)
  0: ObjectId('695cca0aa392d4601b884d96')
interests : Array (2)
  0: "Fintech"
  1: "Startups"
```

6. **$pull** – used to remove values from array fields

```
> db.Attendees.updateOne(
    { name: "Kevin Wong" },
    { $pull: { interests: "Blockchain" } }
  )
```

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
_id: ObjectId('695cd174a392d4601b884d9f')
name : "Kevin Wong"
email : "kevin.wong@startup.io"
type : "Professional"
registered_events : Array (1)
  0: ObjectId('695cca0aa392d4601b884d96')
interests : Array (2)
  0: "Fintech"
  1: "Startups"
```

3.1.4 Delete Operations

CODE	OUTPUT
1. deleteOne() – used to remove one value	
`> db.Attendees.deleteOne({ email: "siti.a@utm.edu.my" })`	`< { acknowledged: true, deletedCount: 1 }`
2. deleteMany() – used to remove many values in one time that meet certain conditions	
`> db.Events.deleteMany({ category: "Finance" })`	`< { acknowledged: true, deletedCount: 2 }`

Advanced Operations

This section will explain how to create indexing, aggregation, and sorting to improve the performance of the database for event management system and to provide the analysis.

4.1 Indexing Implementation

Indexing is implemented to avoid inefficient collection scans and improve query response times.

4.1.1 Single-field Index

CODE	OUTPUT
1. createIndex({index:1})	

`> db.Attendees.createIndex({"registered_events":1})`
`< registered_events_1`
`Atlas atlas-i8djm1-shard-0 [primary] EventManagementDB >`	

This indexing is important because the system frequently queries the Attendees collection to list all participants registered for a specific registered event.	

```
> db.Events.createIndex({"tags":1})
< tags_1
Atlas atlas-i8djm1-shard-0 [primary] EventManagementDB>
```

This indexing is important because users frequently search for events using keywords. This will allows the database to instantly locate all events that has the specific tag without scanning all registered events in the database.

4.1.2 Compound Index

CODE	OUTPUT
1. **createIndex({index:1, index:1})**	

```
> db.Events.createIndex({"category":1, "tickets.price":-1})
< category_1_tickets.price_-1
Atlas atlas-i8djm1-shard-0 [primary] EventManagementDB>
```

This indexing is important because users often filter the events by category and at the same time they want to see the search results sorted by price from highest to lowest, so this index will help sort and filter in a single operation.

```
> db.Events.createIndex({"category":1, "date":1})
< category_1_date_1
Atlas atlas-i8djm1-shard-0 [primary] EventManagementDB>
```

This indexing is important because users might want to filter events by a specific category and list them in ascending order, so this index will help match the category and sort the events by date.

4.2 Aggregation Implementation

Aggregation is used to transform and summarise data for administrative reporting in this project.

CODE	OUTPUT
1. **Total event count by event category by date filtering –** filter all events and then group them by category to show which event types are most active by filtering the date.	

```
> db.Events.aggregate([
    {$match:{date: {$gte: ISODate("2026-04-01")}}},
    {$group:{_id:"$category",totalEvents:{$sum:1}}}
  ])
```

```
< {
    _id: 'Education',
    totalEvents: 2
  }
  {
    _id: 'Technology',
    totalEvents: 1
  }
```

2. **Total attendees by profession –** filter and group the attendees by their profession type to see how many people have registered for the events.

```
> db.Attendees.aggregate([
    {$group: {_id:"$type", totalAttendees:{$sum:1}}}
  ])
```

```
< {
    _id: 'Professional',
    totalAttendees: 3
  }
  {
    _id: 'Student',
    totalAttendees: 3
  }
  {
    _id: 'General',
    totalAttendees: 2
  }
```

3. **Total tags count for each category –** group the categories and calculates the toal number of tags used across all events in that category.

```
> db.Events.aggregate([
    {$group: {_id:"$category", totalTags: {$sum: {$size: "$tags"}}}}
  ])
```

```
< {
    _id: 'Education',
    totalTags: 6
  }
  {
    _id: 'Technology',
    totalTags: 8
  }
```

4. **Total Capacity for each Event –** total up the sum of the capacity of each event

```
> db.Events.aggregate([
    {
      $project: {
        _id:0,
        title:1,
        totalCapacity:{$sum:"$tickets.available_seats"}
      }
    },
    {$sort: { totalCapacity: -1}}])
```

```
< {
    title: 'Global AI & Robotics Summit 2026',
    totalCapacity: 500
  }
  {
    title: 'AI Bootcamp 2026',
    totalCapacity: 200
  }
  {
    title: 'Future of Data Engineering Seminar',
    totalCapacity: 100
  }
  {
    title: 'Cloud-Native Development Workshop',
    totalCapacity: 80
  }
```

4.3 Sorting Implementation

Sorting is implemented to organise the query results for better user experience and reading.

CODE	OUTPUT
1. **Ascending order –** filter and list the events from the oldest to the newest date and displaying only the event title and date	
`> db.Events.find({},{"_id":0, "title":1, "date":1}).sort({"date":1})`	`< { title: 'Cloud-Native Development Workshop', date: 2026-03-15T09:00:00.000Z } { title: 'Future of Data Engineering Seminar', date: 2026-04-10T10:00:00.000Z } { title: 'Global AI & Robotics Summit 2026', date: 2026-05-05T09:00:00.000Z } { title: 'AI Bootcamp 2026', date: 2026-06-01T09:00:00.000Z }`
2. **Descending order** - filter and list the events from the newest to the oldest date and displaying only the event title and date	

```
> db.Events.find({},{"_id":0, "title":1, "date":1}).sort({"date":-1})
```

```
{
  title: 'AI Bootcamp 2026',
  date: 2026-06-01T09:00:00.000Z
}
{
  title: 'Global AI & Robotics Summit 2026',
  date: 2026-05-05T09:00:00.000Z
}
{
  title: 'Future of Data Engineering Seminar',
  date: 2026-04-10T10:00:00.000Z
}
{
  title: 'Cloud-Native Development Workshop',
  date: 2026-03-15T09:00:00.000Z
}
```

4.4 Query Performance

In this project, we analysed the query execution satistics using the explain("executionStats") method to evaluate the efficiency of the indexing strategy. We compared the database performance before and after applying the index on the tags field.

CODE	OUTPUT
1. Performance without index	
`> db.Events.find({"tags":1}).explain("executionStats")`	`>_MONGOSH`
	`executionStats: { executionSuccess: true, nReturned: 0, executionTimeMillis: 0, totalKeysExamined: 0, totalDocsExamined: 4, executionStages: { isCached: false, stage: 'COLLSCAN', filter: { tags: { '$eq': 1 } }, nReturned: 0, executionTimeMillisEstimate: 0, works: 5, advanced: 0, needTime: 4, needYield: 0, saveState: 0, restoreState: 0, isEOF: 1, direction: 'forward', docsExamined: 4 } },`
2. Performance with index	

```
> db.Events.find({"tags":1}).explain("executionStats")
```

Without indexing	With indexing
The execution stage is COLLSCAN means it is Collecion Scan. This shows that MongoDB had to scan every single document in the Events collection to find matches. The document examined is equal to the total number of documents in the collection.	The execution stage changed to IXSCAN means that it is Index Scan. This shows that MongoDB uses B-Tree index structure to jump directly to the index that contains the pointers to the specific documents. The document examined is only the documents that actually matched the query.
Time complexity is O(N) which is linear.	Time complexity is O(log N) which is logarithmic time.

The above comparison data shows that the indexing has reduces the workload of the database. Although collection scanning is good for small datasets, index scanning is important for scalability and bigger datasets. For example, when the Events collection reaches thousands of records, the IXSCAN method will keep the query response time instant and COLLSCAN will eventually slow down.

Security & Limitations

5.1 Basic Access Controls

In this Event Management System, basic access control is considered at a conceptual level rather than fully implemented, as the scope focuses on MongoDB backend operations only. Basic access control is to protect sensitive data stored in the MongoDB database. Different levels of access are conceptually defined for organisers and general users. For instance, organisers are responsible for handling event details and attendee information, while general users are limited to only view event information and register as attendees.

MongoDB provides Role-Based Access Control (RRAC), which can be applied in a real-world implementation to limit read and write permissions for different roles. However, in this project, RRAC was not fully configured, and access control remains a design consideration rather than an enforced mechanism.

5.2 Injection Risk

Injection risk refers to a security issue where database queries behave differently from what the developer intended because the query structure can be manipulated. Although MongoDB does not use SQL, the system is still exposed to NoSQL injection risks if queries are dynamically constructed and external values are interpreted as part of the query logic instead of simple data values. Those manipulations may cause queries to return unintended results or bypass filtering conditions, which can lead to unauthorized data access. However, in this project, the risk is minimal as all database operations are executed directly without user input.

5.3 Database Constraints and Limitations

The flexibility schema design of MongoDB allows data to be stored without strict structural constraints, which is useful for rapid development. However, this flexibility may also cause variations in document structure if data is not carefully managed. Without enforced validation rules, documents within the same collection may differ in format, which can make maintenance and data handling more difficult over time.

Another limitation is that MongoDB does not support traditional relational join operations. When data from multiple collections needs to be retrieved together, aggregation pipelines must be used, which can increase query complexity. In distributed environments, MongoDB applies an eventual consistency approach, meaning updates may not be synchronized across all replicas immediately. This can result in short-term inconsistencies between related collections, representing a trade-off between consistency, performance, and scalability.

Teamwork

This project was completed through effective collaboration and clear task distribution among team members. Each member contributed to different components of the Event Management System to ensure balanced workload and timely completion.

Team Member	Role & Contributions
Brendan Chia Yan Fei [A23CS0211]	• Designed the overall NoSQL database schema for the event management system • Structured core MongoDB collections including Events, Attendees, Organisers, and Venues • Determined appropriate data types and document structures • Contributed to explanations on embedding and referencing strategies
Sabrina Heng Wei Qi [A23CS0265]	• Implemented CRUD operations for multiple collections • Developed query operations using condition operators • Applied projection techniques to retrieve required fields efficiently • Assisted in testing and validating query outputs
Lau Yan Kai	• Implemented single-field and compound indexing strategies.

[A23CS0098]	• Developed aggregation pipelines for analytical queries. • Implemented sorting operations in ascending and descending order • Analysed query performance
Gui Kah Sin [A23CS0080]	• Analysed security considerations including access control and injection risks • Identified system limitations related to schema flexibility and data consistency • Prepared the teamwork and conclusion sections • Reviewed the final report and code structure as well as consistency.

Conclusion

7.1 What Was Learned

Through this project, we gained hands-on experience in designing and implementing a NoSQL database using MongoDB for an event management system. We learned how to model real-world scenarios using collections and documents, and how to apply embedding and referencing strategies appropriately based on data relationships. The project also strengthened our understanding of core CRUD operations and how they are used to manage and manipulate data efficiently in a document-oriented database.

In addition, we learned how indexing and aggregation pipelines can be used to improve query performance and generate meaningful insights from stored data. By analyzing query execution using MongoDB's explain function, we were able to observe the impact of indexing on performance. Overall, this project enhanced our understanding of NoSQL database concepts and provided practical exposure to database design, optimization, and analysis.

7.2 Challenges

One of the main challenges faced during this project was managing the inconsistency data due to MongoDB's flexible schema design. Since MongoDB does not enforce strict schemas by default, careful planning was required to ensure that documents within the same collection followed a consistent structure. Without proper attention or manage, inconsistencies in field names, data types, or missing attributes could occur, which may complicate data retrieval and maintenance.

Another challenge was designing and implementing aggregation pipelines for analytical queries. Aggregation operations require a strong understanding of MongoDB stages such as filtering, grouping, and projecting data. Besides, combining data from multiple collections using aggregation pipelines increased query complexity and required multiple testing iterations to achieve correct and efficient results. In addition, selecting suitable indexing strategies was challenging, as improper indexing could negatively affect performance or consume unnecessary storage.

7.3 Recommendations for Improvement

For future improvements, schema validation rules can be implemented to ensure consistent document structures and reduce the risk of incorrect data insertion. The system can also be extended by integrating an application layer to support real user interaction, such as event registration. Additionally, implementing role-based access control would enhance database security by restricting access to sensitive operations. Further optimization of indexing strategies and expanding aggregation queries for reporting and analytics would improve performance and provide deeper insights for event management.